Using AI to improve your code

Eris :3 (they/them)

October 21, 2026

Using ~~AI~~ Abstract Interpretation to improve your code

Eris :3 (they/them)

October 21, 2026

How other people use Abstract Interpretation to improve your code when you run it

Eris :3 (they/them)

October 21, 2026

I FUCKING HATE AI!!!

I FUCKING HATE AI!!!

Please imagine a video of Sam Altman being blown up by a missile.

I FUCKING HATE AI!!!

Please imagine a video of Sam Altman being blown up by a missile.
These are my thoughts on ai :)

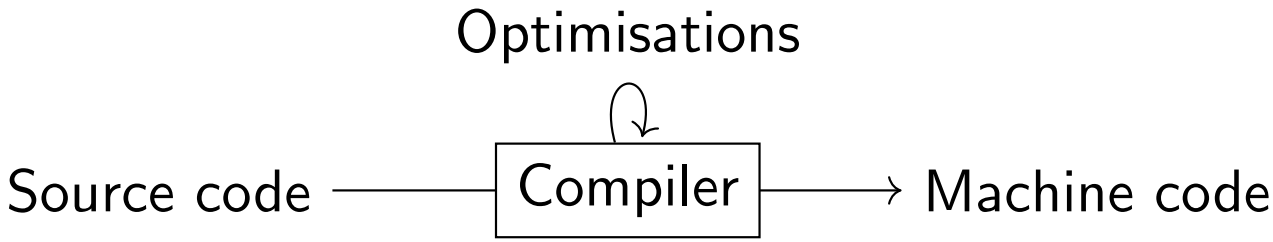
I FUCKING HATE AI!!!

Please imagine a video of Sam Altman being blown up by a missile.
These are my thoughts on ai :) Artificial intelligence will not be mentioned again for the rest of this talk.

Optimising compilers



Optimising compilers



Fibonacci program

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$
$F[0] \leftarrow 0$
$F[1] \leftarrow 1$
$i \leftarrow 2$
while $i < 100$ **do**
    $F[i] \leftarrow F[i-1] + F[i-2]$
    $i \leftarrow i + 1$
end while
return F

Fibonacci program

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$
if $\neg(0 \leq 0 \leq 99 \wedge 0 \leq 1)$ **then**
    explode (out of bounds)
end if
$F[0] \leftarrow 0$
$F[1] \leftarrow 1$
$i \leftarrow 2$
while $i < 100$ **do**
    if $\neg(0 \leq i - 2 \leq 99 \wedge 0 \leq i - 1 \leq 99 \wedge 0 \leq i \leq 99)$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i - 1] + F[i - 2]$
    $i \leftarrow i + 1$
end while
return F

Fibonacci program

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$
if $\neg(0 \leq 0 \leq 99 \wedge 0 \leq 1)$ **then**
    explode (out of bounds)
end if
$F[0] \leftarrow 0$
$F[1] \leftarrow 1$
$i \leftarrow 2$
while $i < 100$ **do**
    if $\neg(0 \leq i - 2 \leq 99 \wedge 0 \leq i - 1 \leq 99 \wedge 0 \leq i \leq 99)$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i - 1] + F[i - 2]$
    $i \leftarrow i + 1$
end while
return F

The bounds check is redundant!

Syntax

Expressions, conditions and statements are defined inductively. Let n be numeric literals and v variables.

$$
\begin{array}{lcl}
e & := & n \mid v \mid e + e \mid e - e \mid e \times e \mid e \div e \\
c & := & \mathrm{true} \mid \mathrm{false} \mid e \leq e \mid e = e \mid c \wedge c \mid c \vee c \mid \neg c \\
s & := & v \leftarrow e \mid s; s \mid \textbf{if } c \textbf{ then } s \textbf{ else } s \textbf{ end if} \\
 & & \mid \textbf{while } c \textbf{ do } s \textbf{ end while} \mid \textbf{explode} \mid \textbf{skip}
\end{array}
$$

Oh wait arrays

Let a be array variables. Then we can extend expressions and statements to include array syntax.

$$\begin{array}{rcl} e & := & \cdots \mid a[e] \\ s & := & \cdots \mid a[e] \leftarrow e \end{array}$$

Semantics

There is a set $\mathcal{E}$ of states (E for environment i think?) and (letting $\mathcal{D} = \mathcal{P}(\mathcal{E})$) an operator $\mathbb{S}[\![s]\!] : \mathcal{D} \to \mathcal{D}$ for each statement s. This operator describes "what the statements do".

Semantics

There is a set $\mathcal{E}$ of states (E for environment i think?) and (letting $\mathcal{D} = \mathcal{P}(\mathcal{E})$) an operator $\mathbb{S}[\![s]\!] : \mathcal{D} \to \mathcal{D}$ for each statement s. This operator describes "what the statements do".
Let's write it out! Let $\mathcal{E} = \mathbb{Z}^V \times A^W$ where V is the set of variables and W is the set of array variables.

Semantics

There is a set $\mathcal{E}$ of states (E for environment i think?) and (letting $\mathcal{D} = \mathcal{P}(\mathcal{E})$) an operator $\mathbb{S}[\![s]\!] : \mathcal{D} \to \mathcal{D}$ for each statement s. This operator describes "what the statements do".
Let's write it out! Let $\mathcal{E} = \mathbb{Z}^V \times A^W$ where V is the set of variables and W is the set of array variables. And A is the set of integer valued arrays... so that would be... umm

$$A = \bigcup_{n \in \mathbb{N}} \{n\} \times \mathbb{Z}^n$$

so like each array has a number telling us its length and then a whole bunch of integers...

Semantics

There is a set $\mathcal{E}$ of states (E for environment i think?) and (letting $\mathcal{D} = \mathcal{P}(\mathcal{E})$) an operator $\mathbb{S}[\![s]\!] : \mathcal{D} \to \mathcal{D}$ for each statement s. This operator describes "what the statements do".
Let's write it out! Let $\mathcal{E} = \mathbb{Z}^V \times A^W$ where V is the set of variables and W is the set of array variables. And A is the set of integer valued arrays... so that would be... umm

$$A = \bigcup_{n \in \mathbb{N}} \{n\} \times \mathbb{Z}^n$$

so like each array has a number telling us its length and then a whole bunch of integers... and then we can define the semantics operator..... that sounds SOOOO annoying!! these arrays are so ugly! i don't wanna

Order theory 1

Definition

A poset is a set P equipped with a partial order $\sqsubseteq$ that's reflexive, antisymmetric and transitive.

(I really hope you've seen this before else this talk will go way too fast)

Order theory 1

Definition

A poset is a set P equipped with a partial order $\sqsubseteq$ that's reflexive, antisymmetric and transitive.

(I really hope you've seen this before else this talk will go way too fast)

Example

For any set S, $(\mathcal{P}(S), \subseteq)$ is a poset.

Order theory 1

Definition

A map $f : P \to Q$ between posets is monotone if whenever $x \sqsubseteq_P y$, $f(x) \sqsubseteq_Q f(y)$.

Order theory 1

Definition

A map $f : P \to Q$ between posets is monotone if whenever $x \sqsubseteq_P y$, $f(x) \sqsubseteq_Q f(y)$.

Proposition

The identity map is monotone. Composites of monotone maps are monotone.

Proof.

If $x \sqsubseteq y$ then $x \sqsubseteq y$.
If $x \sqsubseteq y$ then $f(x) \sqsubseteq f(y)$ then $g(f(x)) \sqsubseteq g(f(y))$. □

Order theory 1

Definition

A map $f : P \to Q$ between posets is monotone if whenever $x \sqsubseteq_P y$, $f(x) \sqsubseteq_Q f(y)$.

Proposition

The identity map is monotone. Composites of monotone maps are monotone.

Proof.

If $x \sqsubseteq y$ then $x \sqsubseteq y$.
If $x \sqsubseteq y$ then $f(x) \sqsubseteq f(y)$ then $g(f(x)) \sqsubseteq g(f(y))$. □

Monotone maps are ordered pointwise, $f \sqsubseteq g$ iff $f(x) \sqsubseteq g(x)$ for all x in the domain.

Approximating semantics

Instead of defining the full semantics, define an approximation set of states $\hat{\mathcal{E}}$ and (letting $\hat{\mathcal{D}} = \mathcal{P}(\hat{\mathcal{E}})$) define an approximation operator $\hat{\mathbb{S}}[\![s]\!] : \hat{\mathcal{D}} \to \hat{\mathcal{D}}$ for each statement s.

Approximating semantics

Instead of defining the full semantics, define an approximation set of states $\hat{\mathcal{E}}$ and (letting $\hat{\mathcal{D}} = \mathcal{P}(\hat{\mathcal{E}})$) define an approximation operator $\hat{\mathbb{S}}[\![s]\!] : \hat{\mathcal{D}} \to \hat{\mathcal{D}}$ for each statement s.
Need a monotone map $\hat{\gamma} : \hat{\mathcal{D}} \to \mathcal{D}$ such that $\mathbb{S}[\![s]\!] \circ \gamma \sqsubseteq \gamma \circ \hat{\mathbb{S}}[\![s]\!]$.

$$\begin{array}{ccc} \hat{\mathcal{D}} & \xrightarrow{\hat{\mathbb{S}}[\![s]\!]} & \hat{\mathcal{D}} \\ {\scriptstyle\gamma}\downarrow & \swarrow\!\!\!\swarrow & \downarrow{\scriptstyle\gamma} \\ \mathcal{D} & \xrightarrow[\mathbb{S}[\![s]\!]]{} & \mathcal{D} \end{array}$$

Approximating semantics

Instead of defining the full semantics, define an approximation set of states $\hat{\mathcal{E}}$ and (letting $\hat{\mathcal{D}} = \mathcal{P}(\hat{\mathcal{E}})$) define an approximation operator $\hat{\mathbb{S}}[\![s]\!] : \hat{\mathcal{D}} \to \hat{\mathcal{D}}$ for each statement s.
Need a monotone map $\hat{\gamma} : \hat{\mathcal{D}} \to \mathcal{D}$ such that $\mathbb{S}[\![s]\!] \circ \gamma \sqsubseteq \gamma \circ \hat{\mathbb{S}}[\![s]\!]$.

$$\begin{array}{ccc} \hat{\mathcal{D}} & \xrightarrow{\hat{\mathbb{S}}[\![s]\!]} & \hat{\mathcal{D}} \\ {\scriptstyle\gamma}\downarrow & \swarrow & \downarrow{\scriptstyle\gamma} \\ \mathcal{D} & \xrightarrow[\mathbb{S}[\![s]\!]]{} & \mathcal{D} \end{array}$$

If you prefer, we need $\hat{\gamma}$ a lax natural transformation viewing semantics opeators as functors from the one object category of statements under sequential composition to the poset-enriched category of posets!

Approximating semantics

Define $\hat{\mathcal{E}} = \mathbb{Z}^V$ (no arrays!) and $\hat{\mathcal{D}} = \mathcal{P}(\hat{\mathcal{E}})$. Then define $\gamma : \hat{\mathcal{D}} \to \mathcal{D}$ by $\gamma(S) = S \times A^W$, giving all possible assignments of array variables. This is clearly monotone!